

P3DX Cross Border Project Task

Scenario

Imagine you're an Indian citizen who studied at a university in Bengaluru, now works at a company in Berlin, and needs their degree verified by a German employer — while their medical history sits with a hospital back home and their tax identity sits with the Indian government. Today, this is a nightmare of PDFs, emails, notarised copies, and portals that don't talk to each other. Each institution trusts its *own* records but has no clean, governed way to trust — or accept — records from another.

Now imagine it *did* work. Your degree, your health record, your identity, your employment history each live in a secure **Locker** that *you* own. When the German employer needs your degree, it doesn't get a photocopy — it opens a **governed connection** to your university's Locker, you **grant consent** for exactly that one purpose, the data flows, and the whole exchange is recorded. Nobody sees more than they're allowed to. Nothing moves without permission. Every hop is accountable.

But the moment you allow this, a hard question appears: who governs it?

Your university operates under Indian data law. The German employer operates under EU (GDPR) rules. The hospital is bound by medical-privacy regulations. These are different **jurisdictions**, different **institutions**, different rulebooks — yet a single request has to cross all of them. You can't just have one global free-for-all; each domain must keep its own rules while still being able to *interoperate* with the others.

This is exactly the problem the Indian Urban Data Exchange models with **Worlds**.

Worlds, Boundaries, and Governance

Data here does not live in one flat space — it lives inside **Worlds**. A World is a **bounded trust and governance domain**, a region that enforces its own rules:

- A **Jurisdiction** (a country, a state) — e.g., *India vs the EU*.
- An **Institution** nested inside a jurisdiction — a University, a Company, a Hospital.
- A **Departmental World** nested even deeper inside an institution.

Worlds are **nested and cross-border**: an Indian Jurisdictional World contains a University World; a separate EU Jurisdictional World contains a Company World. For data to move from the university to the employer, it must cross **both institutional and jurisdictional boundaries** — and each boundary is guarded by a single controlled entry/exit point: a **Gateway**. No agent and no data crosses a World except through its Gateway. The Gateway is where the rules of one World meet the rules of another and a crossing is either permitted, negotiated, or denied.

Where AI Agents Change the Weight of the Problem

Until now we've assumed a *human* is clicking "grant consent." But increasingly the party requesting your data won't be a person — it'll be an **AI agent** acting on someone's behalf. A loan-underwriting agent wants your income proof. A hospital's diagnostic agent wants your prior records. A university's verification agent wants to confirm a credential — automatically, at machine speed, thousands of times a day.

This is what makes governed Worlds essential rather than optional. An autonomous agent can't be trusted with a blanket key to your life. It can only be trusted if **every** request it makes is scoped by an explicit consent, bounded by the World it's allowed into, forced through a Gateway, and permanently recorded — so that "an AI accessed my medical record" is never a mystery but a logged, revocable, purpose-limited **transaction**. Worlds give AI agents a lane to operate in *without* dissolving the trust boundaries humans depend on.

In short: the interesting problems begin when an agent — human *or* AI — from one World needs to enter another World, connect to a Locker across that boundary (often across a national border), and exchange data with parties living under different governance rules. Every such cross-World interaction is **mediated** (through a Gateway), **negotiated** (via consent), and **recorded** (as a transaction).

Your task is to build the **React web dashboard** that makes this multi-World, cross-border, agent-driven system visible, navigable, and operable.

System Semantics (must be understood first)

World

- A bounded trust/governance domain with its own rules.
- Worlds can be **nested**: a Jurisdictional World contains Institutional Worlds (University, Company), which can themselves contain Departmental Worlds.
- Examples: *Jurisdictional World* → *University World*, *Company World* → *Departmental World 1*, *Departmental World 2*.

Gateway

- Every World exposes exactly one **Gateway** — the single controlled entry/exit point.
- No agent or data crosses a World boundary except **through its Gateway**.
- Think of it as border control for a World.

Agent

- A user, admin, or AI agent that resides in a World.
- Has a **role** (e.g., Indian Citizen, Hospital, Admin, Institution) and owns one or more Lockers.

Locker

- A secure data container owned by an agent (as in the current system).
- May publish a **Connection Endpoint** that others can connect to.

Connection Endpoint

- A published point on a Locker/World that agents from other Worlds can request a connection to.

Consent

- Permission granted by a data owner that authorises a specific cross-World / cross-Locker data flow — *what data, for what purpose, for how long, under what conditions*.

Transaction

- A recorded data-exchange event between two or more parties, governed by an active connection + valid consent, passing through the relevant Gateway(s).

Requirement

Design and implement a **React web application** (dashboard) that lets a user explore Worlds, navigate across World boundaries, and manage the connections, consents, and transactions that flow between agents living in different Worlds.

Features to Implement

1. World Navigation (core)

- Visualise the **nested World structure** (Jurisdiction → Institution → Department).
- Let the user **drill into a World** to see the agents, lockers, and gateway that live inside it.
- Make the World boundaries and their Gateways clearly represented in the UI.

2. Agent / Locker Directory

- Inside a World, list the agents and their lockers (see the User Directory reference — Image 2 and 3).
- Provide **search and filtering** (by name, role, World, or locker type).

3. Cross-World Connection Flow

- Let an agent **request entry / a connection into another World** through that World's Gateway.
- Show connection states clearly (e.g., *Requested* → *Pending* → *Established*).

4. Consent & Data-Exchange Management

- Once a connection is established, allow the data owner to **grant / view / revoke consent** governing what is shared across the boundary.
- Show consent terms clearly: data, purpose, duration, conditions.

5. Transactions View

- Display a log/feed of **data-exchange transactions** happening between parties across Worlds — who shared what, with whom, under which consent, through which Gateway.

6. UI/UX

- Responsive desktop layout (a mobile-responsive version is a **plus**).
- Clean, consistent styling and intuitive navigation.

You are encouraged to go **creative** here — this is an evolving idea, so a thoughtful visualisation of Worlds/Gateways, a nice way to represent cross-boundary flows, or your own useful feature will be **appreciated**. Ideating the design through figma first and attaching the link while submitting is a **plus**.

Reference Images

- **Image 1 — Jurisdictional World diagram:** the conceptual model — nested Worlds, agents, lockers, gateways, and connection endpoints (use the legend on the right to map each symbol).
- **Image 2 — "My Lockers" screen:** the current Anumati locker view, for continuity with the existing system's look and semantics.
- **Image 3 — User Directory screen:** reference for how agents/entities can be listed, searched, and "entered."

Expected Outcome

A working React web dashboard where a user can:

- Understand and navigate the **nested World structure**,
- Move across **World boundaries via Gateways**,
- Manage **connections, consents, and cross-World data exchange**,
- and experience a clean, intuitive interface.

We are **not looking for a perfect or complete implementation and not all features are compulsory to get implemented** — we want to see **your vision**, how you model this system, and how much of your own logic and creativity you bring to visualising and prototyping it.

Deliverables

- **GitHub repository link** of your implementation.
 - A **video** demonstrating the running application and walking through the operations.
-

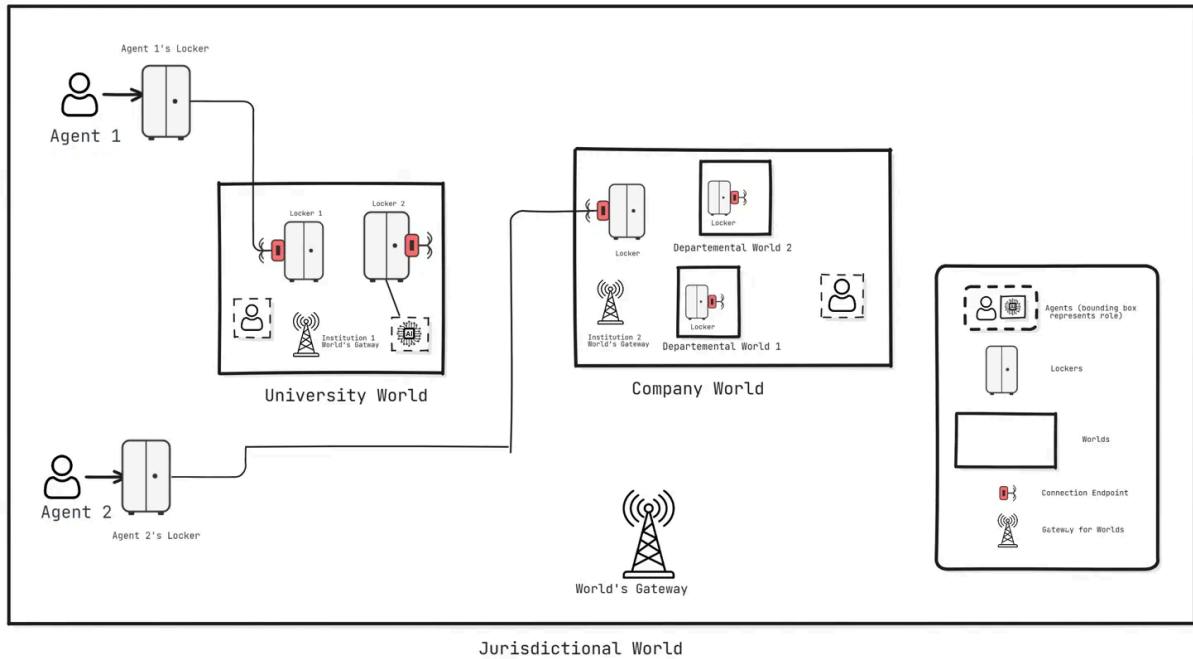


Image 1 (The conceptual diagram of the worlds architecture)

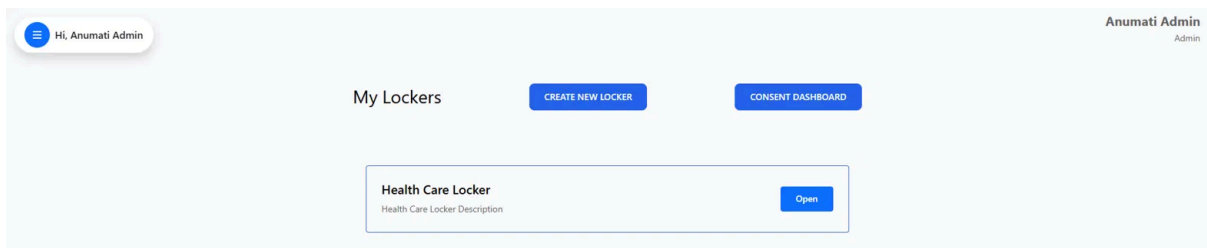


Image 2 (User account inside user its respective locker)

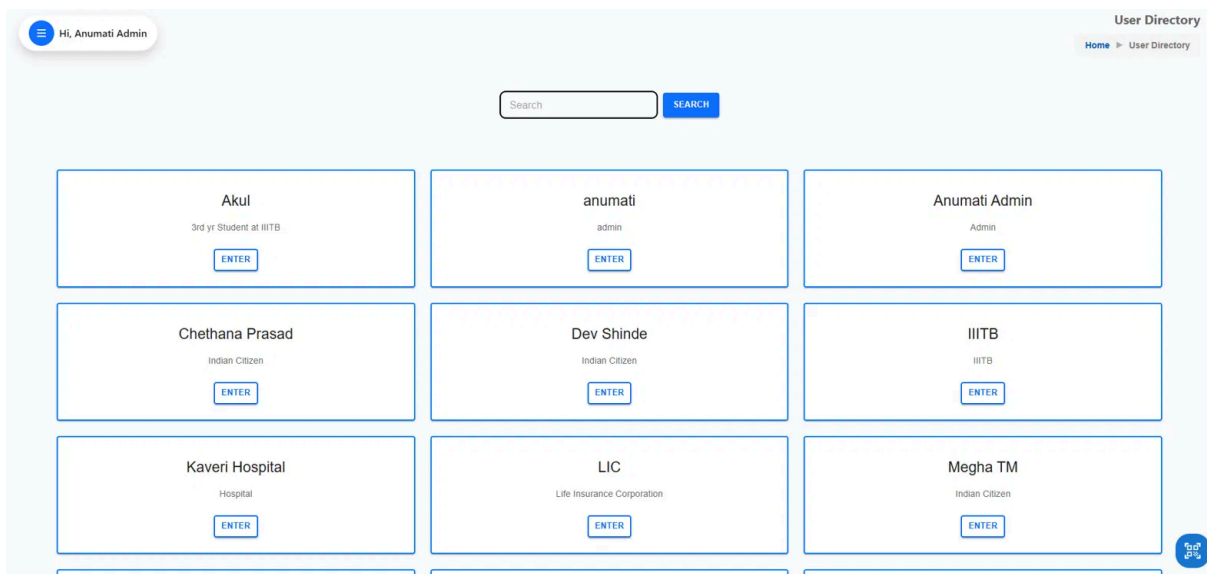


Image 3 - All the different user (inside which there will be diff users and their accounts)